



VIETNAMESE-GERMAN UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE

PROJECT COURSE — FINAL REPORT

MotoVision

Edge-based Pothole Segmentation, Localization, and Reporting

Instructor:

Dr. Vo Bich Hien

Student Team:

Nguyen Gia Thong - 10422117

Le Ba Thai Quan - 10421102

Tran Ngoc Anh Toan - 10422118

Ho Chi Minh City, October 2025

Approval Page

This project report is submitted to the **Department of Computer Science**, Faculty of Engineering, Vietnamese-German University, in partial fulfillment of the requirements for the module **Project Course**.

The report has been examined, reviewed, and approved by the undersigned as satisfying the necessary academic standards.

Approved by:

Dr. Vo Bich Hien

Project Supervisor

Department of Computer Science

Vietnamese-German University

Date: _____

Project Information

Copyright Notice

This report, including its text, figures, tables, and accompanying materials, is the **intellectual property** of the authors and Vietnamese-German University. No part of this publication may be reproduced, stored, transmitted, or distributed without prior written permission, except for brief quotations used for academic purposes with proper citation.

Authors: Nguyen Gia Thong, Le Ba Thai Quan, Tran Ngoc Anh Toan
Course: Project Course
Dept: Computer Science, Faculty of Engineering
University: Vietnamese-German University (VGU)
Date: January 11, 2026

Digital Resources

To facilitate reproducibility, all artifacts are publicly available:

- ◇ **Source Code (GitHub):** MIT License compliant.

<https://github.com/your-repo-link-naouplengibhubseگانlinknha>

- ◇ **Pothole Dataset (Roboflow):**

<https://app.roboflow.com/vietnamese-german-university-nt2sj/pochole/browse?queryText=&pageSize=50&startIndex=0&browseQuery=true>

- ◇ **Field Testing Videos (Google Drive):**

https://drive.google.com/drive/u/0/folders/19MkQ8CwB3M42jMvk_7bPJizlRGqYQnIJ

Contents

1	Introduction	7
1.1	Background	7
1.2	Problem Statement	7
1.3	Scope	8
2	Literature Review	9
2.1	Road Surface Damage and Practical Requirements	9
2.2	Comparative Analysis of Approaches	9
2.3	Segmentation versus Bounding-Box Detection	9
2.4	Edge Deployment Constraints	10
3	Methodology	11
3.1	Edge Pothole Segmentation Pipeline	11
3.2	Model Training & Embedded Export	11
3.3	Real-world Deployment Challenges	12
4	System Design	13
5	Result Analysis and Discussion	17
5.1	Dataset Preparation	17
5.1.1	Annotation Process	17
5.2	Model Training and quantitative Results	18
6	Edge Inference Performance and System-Level Analysis	19
6.1	Performance Context	19
6.2	Post-processing Strategy & Algorithmic Design	20
6.2.1	Algorithmic Motivation	20
6.2.2	The Processing Pipeline	20
6.2.3	Design Considerations and Advantages	21
7	Conclusion	23
8	Limitations and Future Work	24
8.1	Current Limitations	24
8.2	Future Work	24
	References	26
A	Engineering Rationale and Algorithmic Details	27
A.1	Hardware Selection Rationale	27
A.2	Technique Selection & Core Heuristics	28
A.2.1	NCNN Runtime for ARM CPUs	28

- A.2.2 Human-in-the-loop (HITL) Strategy 28
- A.3 Edge-side Algorithmic Implementation 28
 - A.3.1 Asynchronous Pipeline Architecture 28
 - A.3.2 Event Consolidation Logic (Tracking) 28
 - A.3.3 GNSS Smoothing Timestamp Alignment 30
 - A.3.4 Post-processing segmentation mask 32
 - A.3.5 Best-Shot Buffer & Aggregation Logic 33
- A.4 Mobile Cloud Integration 36
 - A.4.1 Mobile UI Strategy 36
 - A.4.2 Cloud Backend (Firebase) 36
- A.5 Bill of Materials (BOM) 36

List of Figures

1	System Architecture	13
2	Data augmentation samples used to improve model robustness against real-world noise.	17
3	Manually Labeled Dataset on Roboflow	18
4	Training metrics over epochs. The convergence of box and mask losses (left) and the steady improvement in mAP (right) indicate successful feature learning. . . .	18
5	Qualitative results on the validation batch. The model successfully delineates pothole boundaries even in challenging lighting.	19
6	Raspberry Pi 4 Model B	27
7	Camera PI Module 3	27
8	GPS BDS ATGM336H	28
9	Decoupled architecture: The MJPEG stream remains smooth for the user even if the Inference thread runs at a lower framerate.	29
10	Visualizing the timestamp alignment strategy. Since camera frames ($30Hz$) occur between discrete GNSS updates ($1Hz$), the system linearly interpolates the precise location based on the exact capture timestamp t_{cap}	30

List of Tables

1	Comparison of Pothole Detection Approaches.	9
2	Deployment Challenges and Mitigation Strategies.	12
3	Communication interfaces in the Edge–Mobile–Cloud system.	16
4	Dataset composition and sources.	17
5	Final Validation Metrics (YOLOv8n-seg).	19
6	Indicative Hardware Cost Estimation.	37

1 Introduction

1.1 Background

Road-surface degradation, particularly **potholes**, constitutes a persistent challenge for urban transportation systems. Beyond mere driving discomfort, potholes significantly increase vehicle maintenance costs, elevate accident risks for vulnerable road users (e.g., motorcyclists), and degrade overall traffic efficiency.

From the perspective of **autonomous and assisted driving**, pothole awareness is a critical component of road-scene understanding, complementing lane detection and obstacle avoidance. However, traditional manual inspection remains costly, slow, and reliant on incomplete citizen reports.

Recent advances in deep learning have made vision-based road damage perception feasible [Maeda et al., 2018]. Nonetheless, deploying such models in real-world scenarios faces substantial constraints: **latency**, **power consumption**, **memory footprint**, and **connectivity intermittency**. Furthermore, environmental variability—such as motion blur, shadows, and lighting changes—poses significant hurdles. These challenges motivate the development of **Edge AI** solutions, where inference occurs locally on embedded hardware to ensure real-time responsiveness without heavy reliance on cloud computation.

1.2 Problem Statement

The core objective is to engineer a practical, end-to-end system capable of the following functions:

1. **Real-time Detection:** Identify potholes from a moving camera using instance segmentation.
2. **Geospatial Logging:** Estimate and record the precise GNSS coordinates of each defect.
3. **Evidence Capture:** Automatically crop and store a representative image of the detected pothole.
4. **Visualization:** Present detections via an Android application featuring a live video stream and map overlay.
5. **Human-in-the-loop:** Enable manual verification and selective upload to the cloud for public reporting and dataset expansion.

Technical Challenge: *A key difficulty lies in achieving robust perception under edge constraints. Potholes exhibit irregular shapes and ambiguous textures often resembling shadows or oil stains. Additionally, asynchronous GNSS measurements complicate accurate geo-tagging, requiring a coherent pipeline that integrates vision inference, temporal filtering, and data management.*

Project Objectives

This project aims to demonstrate a deployable Edge AI prototype with four strategic goals:

Edge Inference	Run a segmentation model on Raspberry Pi 4 under limited compute, targeting usable inference throughput.
Evidence Reporting	Automatically capture pothole imagery and associate detections with GNSS coordinates.
Mobile Interface	Provide a split-screen Android UI (Stream + Map), history browsing, and controlled cloud submission.
Data Flywheel	Support user image contributions to expand training data for future model iterations.

1.3 Scope

The scope of this report is defined by the following boundaries:

Algorithm	A single-class pothole segmentation pipeline utilizing a lightweight <i>YOLOv8-seg</i> variant.
Hardware Deployment	Integration on a specific edge stack: Raspberry Pi 4 + Pi Camera v3 + ATGM336H GNSS , paired with an Android client.
Storage Strategy	A hybrid approach featuring local logging on the edge device and optional cloud upload to Firestore (Firestore Database + Storage) [Google Firebase , a , b].
Design Analysis	Evaluation of algorithmic and architectural choices specifically motivated by edge computing constraints.

2 Literature Review

2.1 Road Surface Damage and Practical Requirements

Pothole detection differs fundamentally from generic object detection. Unlike rigid objects (e.g., cars, pedestrians), potholes are often **texture-defined** rather than shape-defined, possessing weak boundaries and high sensitivity to illumination changes.

Operational systems extend beyond mere detection; they must geo-tag damage and provide actionable evidence for maintenance prioritization [Maeda et al., 2018]. These requirements motivate the adoption of **segmentation-based modeling** combined with robust post-processing logic to handle environmental noise.

2.2 Comparative Analysis of Approaches

Existing literature generally categorizes pothole detection into two streams: Traditional Computer Vision and Deep Learning. Table 1 summarizes the trade-offs between these approaches regarding accuracy and computational cost.

Table 1: Comparison of Pothole Detection Approaches.

Approach	Characteristics & Strengths	Limitations & Challenges
Traditional CV (<i>Handcrafted Features</i>)	Rely on edges, texture descriptors, or stereo/structured light geometry. → Computationally efficient.	Degrade significantly under changing lighting, weather, or camera viewpoints. Lack semantic understanding.
Deep Learning (<i>CNNs, Transformers</i>)	Learn robust feature representations from data. → High generalization across diverse environments.	High compute and memory costs. Deployment on embedded edge devices requires balancing accuracy against latency .

2.3 Segmentation versus Bounding-Box Detection

While bounding-box detectors (e.g., YOLOv8-detect) offer speed, they only localize objects coarsely. For road damage assessment, this project advocates for **Instance Segmentation** due to specific advantages in the edge setting[He et al., 2017].

Key Advantage: Geometric Signal

Instance segmentation provides pixel-level masks that:

- (i) Better capture irregular pothole boundaries compared to rectangular boxes.
- (ii) Enable the estimation of **pothole area** (in pixels) as a direct proxy for severity.

Relevance: This geometric signal allows the use of lightweight heuristics (e.g., mask area thresholds) to filter false positives without needing heavy secondary verification models.

2.4 Edge Deployment Constraints

Deploying vision models on resource-constrained devices (like the Raspberry Pi) introduces system-level constraints that strictly dictate architectural choices:

Compute Limits	Limited CPU/GPU throughput yields low FPS unless highly optimized inference runtimes (e.g., NCNN , TFLite) are utilized.
Thermals & Power	Sustained high-load inference can cause thermal throttling, impacting system reliability and longevity.
Connectivity	Cloud synchronization must be resilient to intermittent networks (4G/WiFi) and should minimize bandwidth usage by uploading only relevant data.
Reliability	False positives are costly when mapped publicly. A human-in-the-loop safeguard is often required to validate machine predictions.

Accordingly, this project adopts an **edge-first** architecture, where critical processing happens locally, with optional and selective cloud synchronization.

Implementation details are shown in Appendix

3 Methodology

3.1 Edge Pothole Segmentation Pipeline

The core perception module utilizes **instance segmentation** via a YOLOv8n-seg model, optimized for the Raspberry Pi 4 using the **NCNN** framework (multi-threaded CPU execution) [He et al., 2017]. The inference pipeline processes each captured frame through the following stages:

Step 1. Inference: The frame undergoes segmentation to generate pixel-level masks.

Step 2. Post-processing: Lightweight logic derives critical metadata:

- Mask contours and visualization overlay.
- **Severity Label:** Computed dynamically based on the mask area (pixels).
- **Event Record:** Encapsulating the image, GNSS coordinates, and timestamp.
- **Alert Trigger:** Immediate notification payload for the Android client.

Strategy: Temporal Tracking & Best-Shot Buffering

To prevent redundant logging of the same pothole across consecutive frames, a **SORT tracker** assigns stable IDs to detections [Bewley et al., 2016]. The system employs a "Best Shot" logic:

- (i) **Buffer:** Maintain a temporal buffer for each tracked ID.
- (ii) **Select:** Identify the frame with the **largest mask area** (highest visibility) that is not truncated by image edges.
- (iii) **Commit:** When the track disappears (patience window expired), the best shot is saved only if it exceeds a minimum area threshold and is not a spatial duplicate.

3.2 Model Training & Embedded Export

A transfer-learning strategy was employed, initializing from a pre-trained **YOLOv8n-seg** checkpoint.

- **Dataset:** 940 labeled images, augmented to ~2,254 samples via geometric and photometric transformations (brightness, blur, horizontal flip).
- **Training Config:** Resolution of 640×640 , trained for 150 epochs. Mosaic augmentation was disabled in the final epochs to enhance convergence on naturalistic features.
- **Export:** The trained PyTorch model was converted to **NCNN** format. This runtime is specifically chosen for its efficient parameter-free implementation on resource-constrained ARM CPUs, eliminating the need for heavy discrete GPUs.

3.3 Real-world Deployment Challenges

Deploying vision models in uncontrolled environments introduces failure modes not fully captured in offline validation. Table 2 summarizes these challenges and the corresponding mitigation strategies adopted in this project.

Table 2: Deployment Challenges and Mitigation Strategies.

Factor	Challenge Description	Project Mitigation
Ambiguity	Shadows, water puddles, and patches resemble potholes.	Use Instance Segmentation (instead of bounding boxes) to verify geometric consistency.
Motion Blur	High vehicle speed and rolling shutter reduce mask fidelity.	Tracking logic (SORT) buffers multiple frames and selects only the clearest "Best Shot".
GNSS Noise	Multipath effects and signal dropout shift location.	Apply Kalman filtering and linear interpolation to smooth coordinate streams [Kalman, 1960 , OpenCV].
Bottleneck	Low inference throughput (~ 4 FPS) on Pi 4 CPU.	Optimized NCNN runtime and Human-in-the-loop verification to filter false positives before upload.

4 System Design

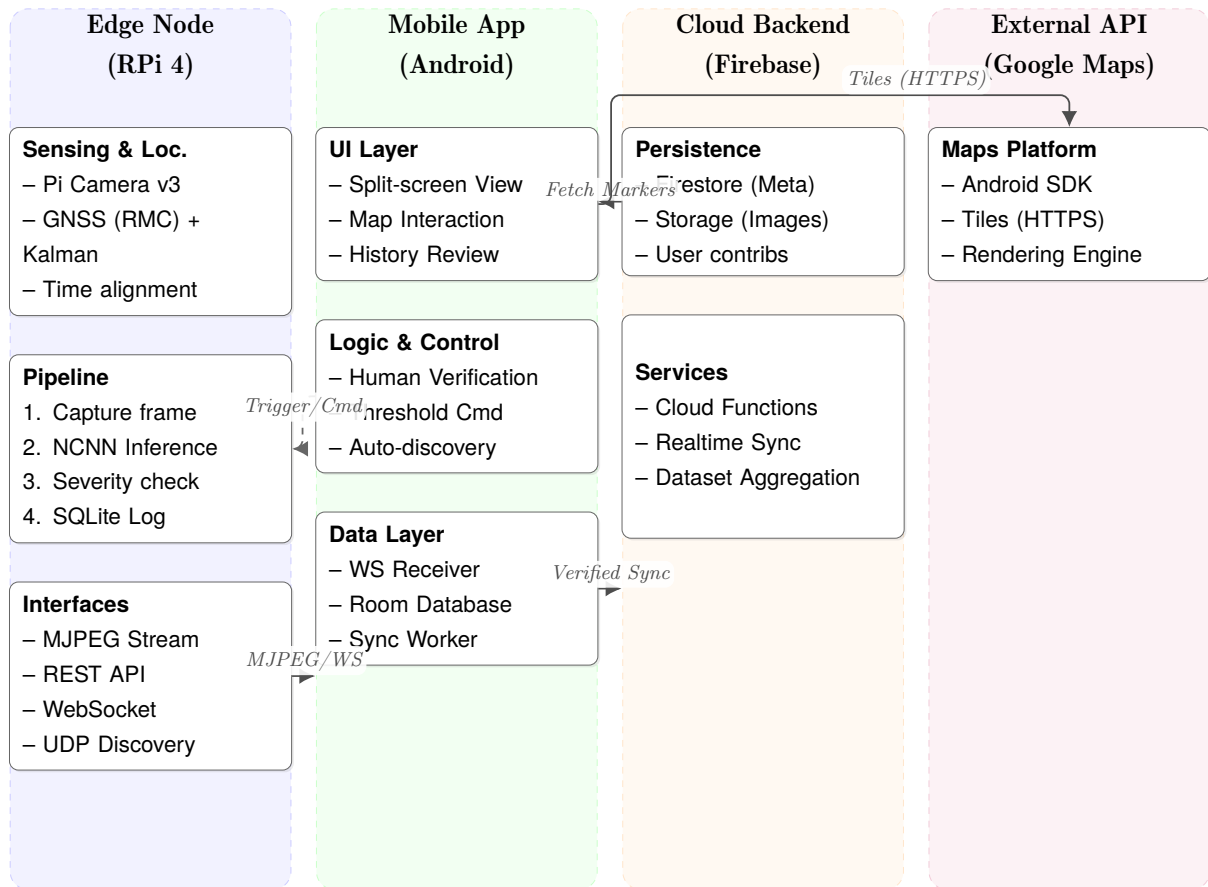


Figure 1: System Architecture

The diagram illustrates the data flow between Edge, Mobile, Cloud, and External services, ensuring clear separation of concerns.

This project implements a distributed **Edge–Mobile–Cloud** architecture. The system is designed to balance on-device performance with user verification and cloud scalability.

Edge Node (Raspberry Pi 4)

The edge node functions as the primary sensing and inference unit, operating under restricted compute and thermal budgets:

- **Vision Sensing:** Pi Camera v3 streams frames to a lightweight video service. The live stream is exposed via an **MJPEG endpoint** (e.g., `/stream.mjpg`) for real-time app visualization.
- **Localization (GNSS):** An ATGM336H module (GPS+BDS) provides geolocation. The system parses NMEA (**RMC**) sentences, converts speed to km/h, and applies a **Kalman filter** to stabilize coordinates. A temporal buffer allows linear interpolation to sync GPS fixes with detection timestamps.
- **Inference Engine:** The segmentation model runs on an **NCNN runtime**, optimized for CPU-efficient execution on the Raspberry Pi.
- **Local Persistence:** Events are logged in **SQLite** (timestamp, lat/lng, speed, severity, image path, sync state) [[SQLite](#)]. A dedicated worker thread manages a bounded save queue to prevent I/O blocking.
- **Connectivity:**
 - **REST API:** Handles log retrieval and upload triggers.
 - **WebSocket:** Broadcasts telemetry (FPS, CPU temp, storage) and receives tuning commands.

Mobile Application (Android)

The Android client acts as a "Human-in-the-loop" interface, combining real-time situational awareness with a verification workflow:

- **Split-Screen UI:**
 - *Upper Panel:* Renders the live MJPEG stream from the Pi.
 - *Lower Panel:* Displays a map using the **Google Maps SDK**, overlaying clustered pothole markers fetched from the cloud [[Google Developers](#)].
- **Verification Workflow:** Users review detection history and manually trigger uploads via the Pi's REST API. This policy ensures only **human-verified data** enters the cloud, minimizing false positives.
- **External Integration:** The mobile interface integrates the **Google Maps SDK for Android** to render the base map and interactive controls. Map tiles are retrieved directly from the Maps platform over HTTPS, while application-specific pothole markers are overlaid from the cloud backend. This separation allows the system to leverage a reliable, high-quality mapping service for visualization while keeping pothole metadata and images under the project's control (Firebase). The Maps API key is configured at the application level (Android manifest metadata) and should be restricted to the Android package signature to prevent unauthorized use.

Cloud Layer (Firebase)

A serverless, maintenance-free backend infrastructure:

- **Firestore (Database):** Stores structured metadata (ID, coordinates, severity, timestamp) in the `potholes` collection.
- **Firebase Storage:** Hosts high-resolution evidence images and user-contributed datasets.
- **Synchronization:** Upon user verification, the Edge node uploads the image to Storage, writes metadata to Firestore, and updates its local SQLite sync state.

Communication Interfaces

Table 3 outlines the protocols used to bridge the distributed components.

Table 3: Communication interfaces in the Edge-Mobile-Cloud system.

Link	Protocol	Endpoint	Primary Purpose
Pi → App	HTTP	/stream.mjpg	Live MJPEG video preview for situational awareness.
Pi ↔ App	WebSocket	:8765	Real-time telemetry (FPS, CPU, Speed) and control commands.
App → Pi	REST	/api/upload	Trigger verified-only upload for selected events.
App → Pi	REST	/api/logs	Fetch local detection history list.
Pi → Cloud	HTTPS	Firebase SDK	Upload image (Storage) and metadata (Firestore) after verification.
App ↔ Cloud	HTTPS	Firestore SDK	Real-time listener for fetching and rendering map markers.
App → Google	HTTPS	Maps Platform	Fetch basemap tiles and render navigation view.

5 Result Analysis and Discussion

5.1 Dataset Preparation

The robustness of the perception system relies heavily on a diverse dataset. As summarized in Table 4, the dataset comprises 940 labeled images aggregated from three distinct sources to cover various road textures and lighting conditions.

Table 4: Dataset composition and sources.

Source Channel	Ratio	Characteristics
Online Repositories	~20%	High-quality, generic road scenes.
Controlled Capture	~40%	Images captured via Pi Camera v3 in static settings.
Real-world Collection	~40%	On-road footage capturing motion blur and shadows.

Preprocessing & Augmentation: To mitigate overfitting and improve generalization against environmental variance (e.g., illumination changes, camera vibrations), we applied geometric and photometric augmentations including *Gaussian blur*, *brightness adjustment*, and *horizontal flips*. Augmentations were applied **only to the training split**, expanding the training set to 1,971 images (and 2,254 images in total including Val/Test).



(a) Original Image



(b) Augmented Sample

Figure 2: Data augmentation samples used to improve model robustness against real-world noise.

5.1.1 Annotation Process

Data annotation was performed on a per-pixel segmentation basis using **Roboflow** [Roboflow]. The base dataset (940 labeled images) was split in Roboflow into Train/Val/Test = 70/20/10. Augmentations were applied **only to the training split**, expanding the training set to 1,971 images, while validation and test remained 189 and 94 images, respectively (total export: 2,254).

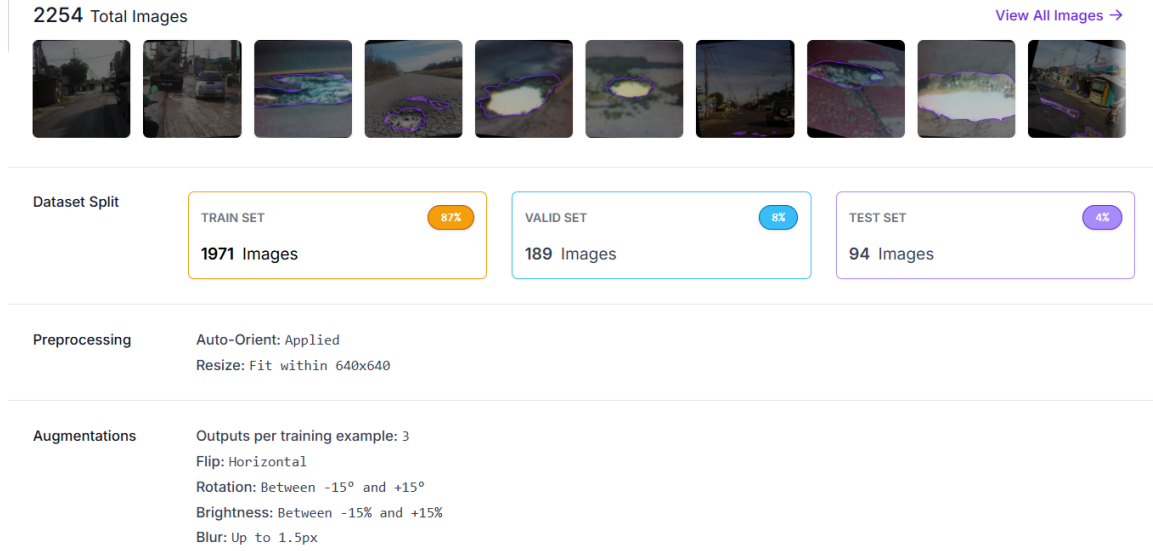


Figure 3: Manually Labeled Dataset on Roboflow

5.2 Model Training and quantitative Results

The segmentation network was trained using a transfer-learning approach initialized from the **YOLOv8n-seg** baseline. Training was conducted on the **Google Colab** platform, utilizing an NVIDIA T4 GPU accelerator.

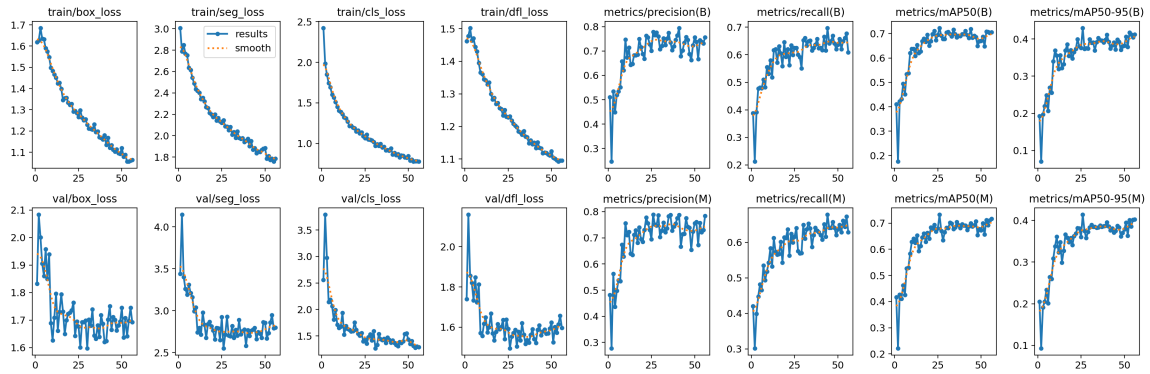


Figure 4: Training metrics over epochs. The convergence of box and mask losses (left) and the steady improvement in mAP (right) indicate successful feature learning.

Performance Metrics: Table 5 details the final performance on the validation set. The model achieved a mask mAP@50 of **0.726**, which is substantially higher than the bounding box performance (0.521).

Table 5: Final Validation Metrics (YOLOv8n-seg).

Task Type	Precision (P)	Recall (R)	mAP ₅₀	mAP ₅₀₋₉₅
Bounding Box	0.560	0.483	0.521	0.342
Segmentation Mask	0.768	0.636	0.726	0.413

Discussion: The results indicate that the segmentation head provides high-quality localization comparable to bounding boxes, validating the use of mask area for severity estimation. The primary performance gap arises from extreme motion blur and domain shifts in asphalt texture, which are common in the *Real-world Collection* subset.

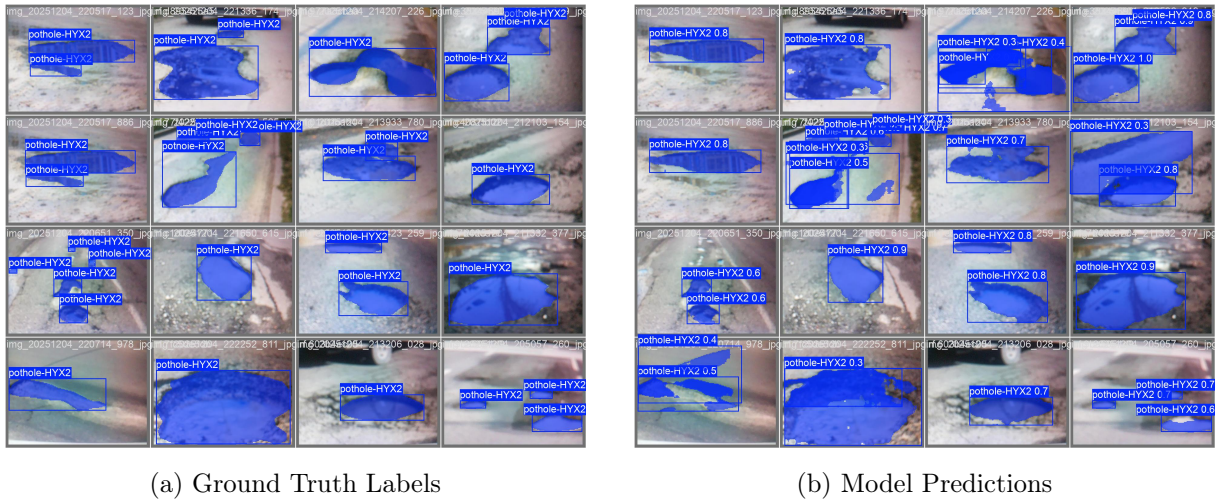


Figure 5: Qualitative results on the validation batch. The model successfully delineates pothole boundaries even in challenging lighting.

6 Edge Inference Performance and System-Level Analysis

6.1 Performance Context

In the context of this pothole detection project, this subsection evaluates the **edge inference throughput** and **user-facing responsiveness**. Currently, the Raspberry Pi 4 achieves an end-to-end throughput of approximately **4 FPS** when executing the NCNN segmentation model on the CPU.

System Viability Analysis

While 4 FPS is below standard real-time video rates (30 FPS), the system remains operationally effective for two reasons:

- (i) **Event-based Objective:** The goal is discrete *event detection* (locating a pothole) rather than dense, per-frame continuous tracking (like autonomous braking).
- (ii) **Intelligent Sampling:** The *tracking-and-buffering* logic effectively aggregates information over time, selecting only the most representative frames for logging, thus mitigating the impact of low frame rates.

6.2 Post-processing Strategy & Algorithmic Design

This section details the pipeline responsible for converting raw, noisy segmentation outputs into reliable, actionable event logs. The design prioritizes computational efficiency and data integrity under limited throughput conditions.

6.2.1 Algorithmic Motivation

Raw model predictions are inherently noisy. Without robust post-processing, the system faces specific challenges:

Redundancy A single pothole appearing in 10 consecutive frames would generate 10 duplicate database entries.

Truncation Potholes at the image edge are often cut off, leading to inaccurate severity estimation.

Jitter GNSS coordinates fluctuate due to sensor noise, causing mapping errors.

6.2.2 The Processing Pipeline

To address these difficulties, the system implements a multi-stage logic pipeline:

- 1. SORT Tracking** In our implementation, each segmentation mask is converted to a tight bounding box for tracking. Associations are computed using IoU-based matching, enabling a lightweight SORT-style identity assignment under CPU constraints.
- 2. Best-Shot Buffering** Instead of saving the first detection, the system maintains a temporary buffer for each Track ID. It dynamically updates and retains only the frame with the **largest mask area** (clearest view) that does not touch the image boundary.
- 3. Edge Rejection** Filters out masks that intersect with the camera frame borders, as these partial detections yield unreliable severity calculations.

4. Duplicate Suppression Enforces a spatiotemporal radius check. Even if tracking fails, a new event is blocked if it falls within X meters and T seconds of a recently logged pothole.

GNSS Stabilization: Parallel to vision processing, the location stream undergoes **Kalman filtering** to reduce jitter in latitude/longitude. A time-indexed buffer allows the system to interpolate and retrieve the precise coordinate corresponding to the exact timestamp of the "Best Shot" capture.

6.2.3 Design Considerations and Advantages

This section outlines the mathematical formulations and logical heuristics employed to bridge the gap between raw sensor data and actionable pothole events.

Mathematical Framework

1. Severity Estimation via Mask Area

Let $M \in \{0,1\}^{H \times W}$ be the binary segmentation mask predicted by the model. The normalized area ratio r is derived as:

$$A_{\text{mask}} = \sum_{i=1}^H \sum_{j=1}^W M_{ij}, \quad \text{and} \quad r = \frac{A_{\text{mask}}}{H \cdot W} \quad (1)$$

To categorize severity efficiently, we apply a thresholding function $\mathcal{S}(r)$:

$$\mathcal{S}(r) = \begin{cases} \text{High} & \text{if } r > 0.03 \\ \text{Medium} & \text{if } 0.01 < r \leq 0.03 \\ \text{Low} & \text{otherwise} \end{cases} \quad (2)$$

Advantage: This heuristic is computationally minimal and provides an **image-space proxy** of pothole extent/visibility. However, physical severity depends on camera viewpoint and distance; thus, this score is used for **relative prioritization** rather than metric depth estimation.

2. GNSS Synchronization (Linear Interpolation)

Given two consecutive smoothed GNSS fixes $(t_1, \mathbf{p}_1)$ and $(t_2, \mathbf{p}_2)$ where $\mathbf{p} = [\text{lat}, \text{lng}]^\top$, the location $\mathbf{p}(t)$ for a frame captured at time $t \in [t_1, t_2]$ is estimated by:

$$\alpha = \frac{t - t_1}{t_2 - t_1}, \quad \mathbf{p}(t) = (1 - \alpha)\mathbf{p}_1 + \alpha\mathbf{p}_2 \quad (3)$$

Advantage: This aligns location data with frame timestamps, mitigating spatial errors caused by the asynchronous update rates of camera (30Hz) and GNSS (1Hz).

Operational Heuristics Beyond mathematical formulations, the system enforces strict logic to ensure data quality and reliability:

Spatiotemporal Filtering To prevent the "flooding" of duplicate events (e.g., detecting the same pothole in multiple frames), a cooldown mechanism is enforced. A new detection is rejected if:

$$\text{dist}(\mathbf{p}_{\text{new}}, \mathbf{p}_{\text{saved}}) < R_{\text{thresh}} \quad \text{AND} \quad |t_{\text{new}} - t_{\text{saved}}| < T_{\text{cooldown}}$$

For small local neighborhoods ($R_{\text{thresh}} \approx 5m$), Euclidean distance serves as an effective and inexpensive approximation for geodetic distance.

Human-in-the-loop (HITL)

The system architecture intentionally decouples *Detection* from *Reporting*.

- **Local Logging:** All detections are saved locally on the device for review.
- **Selective Upload:** Only incidents verified by the user via the Android app are transmitted to the cloud.

This design acts as a final quality gate, filtering out false positives (e.g., shadows, oil stains) that are common in edge-based computer vision.

7 Conclusion

This project successfully designed and implemented **MotoVision**, an end-to-end Edge AI solution addressing the critical challenge of automated road-surface monitoring. By integrating deep learning inference on embedded hardware with a mobile-centric verification workflow, the system bridges the gap between theoretical models and practical, real-world deployment.

The development process and experimental results highlight three significant achievements:

- 1. Feasible Edge Deployment:** The prototype demonstrates that computationally intensive tasks—specifically instance segmentation—can be executed effectively on the **Raspberry Pi 4** without discrete accelerators. The utilization of the **NCNN** runtime was pivotal, enabling the CPU to maintain functional inference throughput while managing thermal constraints.
- 2. Robust System Integration:** Beyond the AI model, the project successfully engineered a coherent sensing pipeline. The integration of **SORT tracking** to prevent redundant logging, combined with **Kalman filtering** for GNSS stabilization, solves two of the most persistent issues in mobile mapping: data duplication and location jitter. The resulting "Best Shot" logic ensures that only high-quality evidence is preserved.
- 3. Human-Centric Reliability:** Recognizing that AI models are prone to false positives in uncontrolled environments, the incorporation of a **Human-in-the-loop** workflow via the Android application serves as a crucial quality control layer. This design choice safeguards the integrity of the cloud dataset, making the system viable for community-driven reporting.

In summary, this report validates that a distributed **Edge–Mobile–Cloud** architecture offers a scalable and cost-effective alternative to traditional road inspection vehicles. The established framework provides a solid foundation for future enhancements in autonomous infrastructure monitoring and smart city data collection.

8 Limitations and Future Work

8.1 Current Limitations

Despite functioning as a complete end-to-end prototype, the current system exhibits inherent constraints primarily dictated by hardware and data limitations:

- | | |
|-------------------------------|---|
| Compute Bottleneck | Inference on the Raspberry Pi 4 CPU is capped at ~ 4 FPS. This throughput restricts responsiveness at high vehicle speeds (>30 km/h) and reduces the temporal redundancy needed for robust tracking. |
| Localization Precision | Standalone GNSS modules are subject to meter-level errors, particularly in urban canyons (multipath effects). This degrades the precise mapping of potholes required for maintenance crews. |
| Data Diversity | While augmented, the dataset ($\sim 2,254$ samples) is modest in scale. It may not fully generalize across diverse asphalt textures, extreme weather conditions (heavy rain/glare), or varying camera mounting angles. |
| Depth Estimation | The system uses mask area as a proxy for severity. It lacks true 3D depth perception (e.g., stereo vision or LiDAR), which is a critical factor in assessing the actual danger level of a pothole. |

8.2 Future Work

To transition from a prototype to a deployable product, future development will focus on the following strategic roadmap:

Development Roadmap

1. **Hardware Acceleration:** Migrate inference to dedicated edge accelerators (e.g., *Google Coral TPU* or *NVIDIA Jetson Nano*) and apply **INT8 quantization**. This aims to achieve real-time performance (20+ FPS) and reduce power consumption.
2. **Sensor Fusion:** Integrate IMU data and Map-matching algorithms to correct GNSS drift, ensuring lane-level accuracy even in signal-denied environments.
3. **Multi-class Taxonomy:** Extend the model to detect a broader range of road damages, including longitudinal cracks, patches, and sunken manholes, providing richer actionable data.
4. **Active Learning Pipeline:** Implement an automated loop where high-confidence user uploads are verified and fed back into the training set, continuously improving model robustness over time.
5. **Large-scale Validation:** Conduct field tests on pre-surveyed routes with ground truth measurements to quantitatively benchmark Precision/Recall, localization error (meters), and system mean-time-between-failures (MTBF).

References

- Hiroya Maeda, Yoshihide Sekimoto, Toshikazu Seto, Takehiro Kashiya, and Hiroshi Omata. Road damage detection using deep neural networks with images captured through a smartphone. *arXiv preprint arXiv:1801.09454*, 2018. URL <https://arxiv.org/abs/1801.09454>.
- Google Firebase. Firebase documentation: Cloud storage. Online documentation, a. URL <https://firebase.google.com/docs/storage>. Accessed: 2025-10.
- Google Firebase. Firebase documentation: Cloud firestore security rules. Online documentation, b. URL <https://firebase.google.com/docs/firestore/security/get-started>. Accessed: 2025-10.
- Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. Mask r-cnn. *arXiv preprint arXiv:1703.06870*, 2017.
- Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. *arXiv preprint arXiv:1602.00763*, 2016.
- R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960. doi: 10.1115/1.3662552.
- OpenCV. Opencv documentation: Kalmanfilter class. Online documentation. URL https://docs.opencv.org/4.x/dc/d2c/classcv_1_1KalmanFilter.html. Accessed: 2025-10.
- SQLite. Sqlite documentation. Online documentation. URL <https://www.sqlite.org/docs.html>. Accessed: 2025-10.
- Google Developers. Google maps sdk for android documentation. Online documentation. URL <https://developers.google.com/maps/documentation/android-sdk/overview>. Accessed: 2025-10.
- Roboflow. Roboflow documentation. Online documentation. URL <https://docs.roboflow.com/>. Accessed: 2025-10.

A Engineering Rationale and Algorithmic Details

This appendix delineates the engineering decisions and algorithmic strategies employed to balance the trade-offs between cost, computational power, and system reliability.

A.1 Hardware Selection Rationale

The hardware stack was chosen to ensure reproducibility and affordability while maintaining sufficient performance for computer vision tasks.

Raspberry Pi 4

Selected as the central edge node due to its widespread availability and support for a full Linux stack. Its 4-core CPU is sufficient for lightweight inference when optimized, avoiding the need for expensive dedicated AI accelerators.

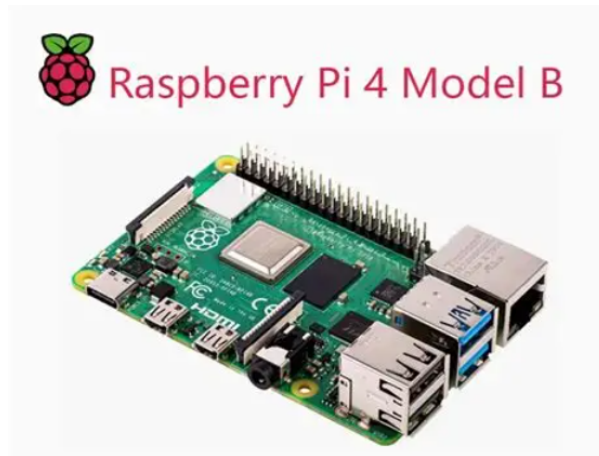


Figure 6: Raspberry Pi 4 Model B

fig:hw_pi4

Pi Camera Module v3

Chosen for its native CSI (Camera Serial Interface) integration. Unlike USB webcams, CSI offers high-bandwidth direct memory access, reducing CPU overhead during frame capture.



Figure 7: Camera PI Module 3

ATGM336H GNSS

A dedicated hardware GNSS module provides outdoor localization strictly independent of the mobile phone, ensuring continuous spatiotemporal logging.



Figure 8: GPS BDS ATGM336H

A.2 Technique Selection & Core Heuristics

A.2.1 NCNN Runtime for ARM CPUs

To achieve usable framerates on the Pi 4 CPU, the model is deployed using **NCNN**. This high-performance framework is specifically optimized for ARM architectures (NEON instructions), offering significantly lower latency compared to standard PyTorch or ONNX Runtime on edge devices.

A.2.2 Human-in-the-loop (HITL) Strategy

Recognizing the visual ambiguity of road damages (shadows vs. holes), the system treats model outputs as *candidates*. The mandatory user verification step acts as a "Gatekeeper," preventing false positives from polluting the public cloud dataset.

A.3 Edge-side Algorithmic Implementation

A.3.1 Asynchronous Pipeline Architecture

To prevent the UI from freezing during heavy inference, the system decouples the pipeline into separate threads.

A.3.2 Event Consolidation Logic (Tracking)

Naively logging every frame creates redundant data. The system employs a **Track-and-Buffer** strategy:

- 1. ID Assignment:** A tracker assigns a unique ID to spatially consistent detections across frames.

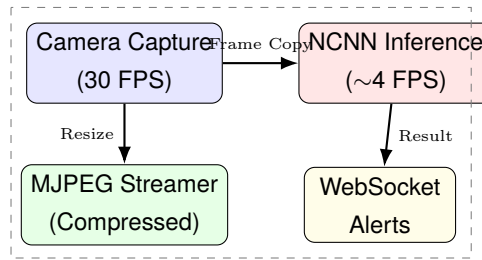


Figure 9: Decoupled architecture: The MJPEG stream remains smooth for the user even if the Inference thread runs at a lower framerate.

2. **Best-Shot Selection:** For each ID, a buffer retains the frame with the *highest confidence* and *largest mask area*.
3. **Commit:** When the object leaves the frame (Patience Window expired), only the "Best Shot" is saved to the database.

```

def compute_iou(a, b):
    """ Calculates Intersection over Union (IoU) between box A and B. """
    x1 = max(a[0], b[0]); y1 = max(a[1], b[1])
    x2 = min(a[2], b[2]); y2 = min(a[3], b[3])

    inter = max(0, x2-x1) * max(0, y2-y1)
    areaA = max(0, a[2]-a[0]) * max(0, a[3]-a[1])
    areaB = max(0, b[2]-b[0]) * max(0, b[3]-b[1])

    union = areaA + areaB - inter
    return inter/union if union > 0 else 0

def match_det_to_track(track_box, dets, thres=0.1):
    """ Greedy matching: Assigns detection with highest IoU to the track. """
    best = None
    best_iou = 0
    for d in dets:
        iou = compute_iou(track_box, d["box"])
        if iou > best_iou:
            best_iou = iou; best = d

    # Reject assignment if overlap is too small (Drift/Occlusion)
    if best_iou < thres: return None
    return best

def build_tracker_input(dets):
    """ Normalizes input format for the SORT tracker [x1, y1, x2, y2, conf] """
    arr = []
    for d in dets:
        x1, y1, x2, y2 = d["box"]
        conf = float(np.clip(d["conf"], 0.0, 1.0))

```

```
arr.append([x1, y1, x2, y2, conf])
return np.array(arr, dtype=np.float32)
```

Listing 1: IoU Calculation & Association Logic

Implementation Logic:

- ◇ **Association Strategy:** Although SORT tracks bounding boxes, the system must associate pixel masks to these tracks. We use `compute_iou` as the similarity metric.
- ◇ **Greedy Matching:** For each existing track, we search for the detection with the highest overlap.
- ◇ **Drift Prevention:** The `thres` parameter rejects matches where IoU is too low (< 0.1), preventing the tracker from incorrectly attaching to a pothole that has drifted significantly or is occluded.
- ◇ **Data Standardization:** `build_tracker_input` ensures confidence scores are strictly clamped $[0, 1]$ before entering the Kalman filter.

A.3.3 GNSS Smoothing Timestamp Alignment

Since GNSS updates (1Hz) are asynchronous to video frames (30Hz), we employ a time-indexed buffer.

- **Smoothing:** A Kalman Filter is applied to raw latitude/longitude to reduce jitter.
- **Interpolation:** When an event occurs at t_{event} , the location is linearly interpolated between GNSS fixes at t_{prev} and t_{next} .

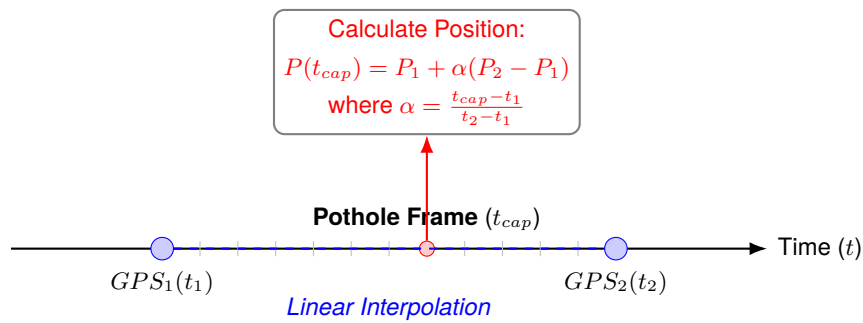


Figure 10: Visualizing the timestamp alignment strategy. Since camera frames (30Hz) occur between discrete GNSS updates (1Hz), the system linearly interpolates the precise location based on the exact capture timestamp t_{cap} .

Algorithm Implementation details:

```
def get_location_at(self, query_timestamp):  
    """  
    Interpolates latitude/longitude for a specific video frame timestamp.  
    """  
    t1, t2 = self.prev_fix['time'], self.curr_fix['time']  
  
    # Fallback: If data is stale (>1.0s) or invalid, return latest known ↵  
    fix  
    if t2 == 0 or t1 == 0 or (t2 - t1) <= 0 or query_timestamp > t2 + 1.0:  
        return self.lat, self.lng  
  
    # Calculate linear ratio based on time  
    ratio = max(0.0, min(1.0, (query_timestamp - t1) / (t2 - t1)))  
  
    # Linear Interpolation formula  
    i_lat = self.prev_fix['lat'] + (self.curr_fix['lat'] - self.prev_fix['↵  
    lat']) * ratio  
    i_lng = self.prev_fix['lng'] + (self.curr_fix['lng'] - self.prev_fix['↵  
    lng']) * ratio  
  
    return i_lat, i_lng
```

Listing 2: GNSS Linear Interpolation Logic to align location with frame timestamp.

Key Implementation Concepts:

- **Frequency Mismatch:** GNSS updates (1Hz) are asynchronous with the Camera (30Hz). Simply fetching the "latest" GPS fix would introduce a temporal offset error.
- **Interpolation Strategy:** The system buffers the two most recent fixes (`prev_fix`, `curr_fix`) and calculates a `ratio` representing the frame's relative position in time between them.
- **Fallback Condition:** To prevent system crashes or wild extrapolations, if the GNSS data is too old (>1.0s) or insufficient (missing 2 points), the system safely falls back to the latest known position.


```
class GPSKalmanFilter:
    def __init__(self):
        # Initialize OpenCV Kalman Filter: 4 dynamic params, 2 measurement↵
        params
        self.kf = cv2.KalmanFilter(4, 2)

        # Measurement Matrix (H): Mapping state to measurement
        self.kf.measurementMatrix = np.array([[1, 0, 0, 0],
                                              [0, 1, 0, 0]], np.float32)

        # Transition Matrix (F): Predicting next state based on physics (↵
        velocity)
        self.kf.transitionMatrix = np.array([[1, 0, 1, 0],
                                              [0, 1, 0, 1],
                                              [0, 0, 1, 0],
                                              [0, 0, 0, 1]], np.float32)

        # Covariance Matrices (Noise tuning)
        self.kf.processNoiseCov = np.eye(4, dtype=np.float32) * 1e-4
        self.kf.measurementNoiseCov = np.eye(2, dtype=np.float32) * 1e-2
        self.is_initialized = False

    def update(self, lat, lng):
        measurement = np.array([[np.float32(lat)], [np.float32(lng)]]])

        if not self.is_initialized:
            # Set initial state
            self.kf.statePre = np.array([[lat], [lng], [0], [0]], np.↵
            float32)
            self.kf.statePost = np.array([[lat], [lng], [0], [0]], np.↵
            float32)
            self.is_initialized = True
            return lat, lng

        # Prediction & Correction steps
        self.kf.predict()
        corrected = self.kf.correct(measurement)
        return float(corrected[0][0]), float(corrected[1][0])
```

Listing 3: Python implementation of the Kalman Filter for GNSS smoothing using OpenCV.

A.3.4 Post-processing segmentation mask

```
# 1. Retrieve and Validate Mask
```

```
mask = best_det["mask"]
x1, y1, x2, y2 = best_det["box"]

if mask is None or mask.size == 0: continue

# Binarize mask if necessary
if mask.dtype != np.uint8:
    mask = (mask > 0.5).astype(np.uint8) * 255

# 2. Resize to Original Frame Dimensions
# Using Nearest Neighbor to preserve sharp binary edges
if mask.shape[:2] != (H, W):
    mask = cv2.resize(mask, (W, H), interpolation=cv2.INTER_NEAREST)

# 3. Calculate Metrics
mask_area = cv2.countNonZero(mask)

# Check if pothole is truncated at image border (5px margin)
is_touching_edge = (x1 < 5 or y1 < 5 or x2 > W-5 or y2 > H-5)

# 4. Assign Severity Label based on Area
severity = "Low"
if mask_area > AREA_THRES_HIGH:
    severity = "High"
elif mask_area > AREA_THRES_MED:
    severity = "Medium"
```

Listing 4: Mask Analysis & Severity Estimation

Implementation Logic:

- ◇ **Mask Resizing:** The raw mask output is resized to match the original frame resolution (W, H) using `cv2.INTER_NEAREST`. We avoid linear interpolation to preserve the sharp binary boundaries of the pothole and minimize CPU usage on the Raspberry Pi.
- ◇ **Area Quantification:** `mask_area` counts the exact number of pixels belonging to the pothole. This serves as a more accurate severity proxy than bounding-box area, which often includes healthy road pavement.
- ◇ **Edge Rejection Strategy:** The `is_touching_edge` flag detects if the bounding box intersects the image border (within 5 pixels). Potholes at the edge are often truncated (partially visible), leading to underestimated severity. These frames are flagged to be *deprioritized* by the "Best-Shot" selector in favor of centered views.
- ◇ **Severity Classification:** The final label ("High", "Medium", "Low") is derived by comparing the pixel count against calibrated thresholds.

A.3.5 Best-Shot Buffer & Aggregation Logic

```
# Logic to update the "Best Shot" for an active Track ID (tid)

if tid not in track_buffer:
    # Initialize new track entry
    track_buffer[tid] = {
        "best_frame": frame.copy(),
        "max_area": mask_area,
        "lat": curr_lat, "lng": curr_lng, "speed": curr_speed,
        "severity": severity,
        "timestamp": datetime.now().strftime("%Y%m%d_%H%M%S"),
        "lost_frames": 0,
        "is_complete": not is_touching_edge # Flag: is the pothole fully ↔
        visible?
    }
else:
    # Reset patience counter since object is detected
    track_buffer[tid]["lost_frames"] = 0
    prev = track_buffer[tid]

    should_update = False

    # Priority 1: Upgrade from Partial (Edge-touching) to Complete view
    if (not prev["is_complete"]) and (not is_touching_edge):
        should_update = True

    # Priority 2: If completeness is equal, chose the larger mask (closer view↔)
    elif (prev["is_complete"] == (not is_touching_edge)) and \
        (mask_area > prev["max_area"]):
        should_update = True

    if should_update:
        track_buffer[tid].update({
            "best_frame": frame.copy(),
            "max_area": mask_area,
            "severity": severity,
            "lat": curr_lat, "lng": curr_lng,
            "is_complete": not is_touching_edge
        })
```

Listing 5: Best-Shot Buffer & Aggregation Logic

Implementation Logic:

- ◇ **Aggregation Objective:** Since a single pothole appears in multiple consecutive video frames, logging every detection would cause data redundancy. This logic ensures we save only **one** optimal representative image per pothole.

- ◇ **Completeness Priority:** The algorithm prioritizes "Complete" views over "Truncated" ones. If the current best shot touches the image edge (potentially cutting off part of the pothole), it is immediately replaced by any new frame where the pothole is fully inside the bounds.
- ◇ **Visibility Optimization:** Among frames with the same completeness status, the system selects the one with the **largest mask area**. This typically corresponds to the moment the vehicle is closest to the pothole, providing the clearest texture details.
- ◇ **Lazy Commit:** The data in `track_buffer` is temporary. The final database write only occurs when `lost_frames` exceeds a patience threshold (indicating the pothole has left the camera view).

```
\begin{lstlisting}[language=Python, numbers=None, frame=None]
# Iterate through all active tracks to check for disappearance
for track_id in list(track_buffer.keys()):
    if track_id not in current_frame_ids:
        track_buffer[track_id]["lost_frames"] += 1

    # 1. Check Patience: Only commit if object is lost for N frames
    if track_buffer[track_id]["lost_frames"] > PATIENCE_LIMIT:
        data = track_buffer[track_id]

    # 2. Noise Filter: Ignore too small detections
    if data["max_area"] > MIN_SAVE_AREA:
        is_duplicate = False

    # 3. Spatial Deduplication (Check against recent history)
    for la, lo, t in recent_saved:
        # Approx Euclidean distance for speed
        dist = ((data["lat"]-la)**2 + (data["lng"]-lo)**2)**0.5
        if dist < GPS_RADIUS:
            is_duplicate = True
            break

    # 4. Async Save (Non-blocking push)
    if not is_duplicate:
        save_queue.put_nowait((
            data["best_frame"],
            data["lat"], data["lng"], data["speed"],
            data["timestamp"], data["severity"]
        ))
        # Add to local history for future dedup checks
        recent_saved.append((data["lat"], data["lng"], now))

    # 5. Cleanup: Remove from buffer to free memory
    del track_buffer[track_id]
```

Listing 6: Event Trigger & Deduplication Logic

Implementation Logic:

- ◇ **Patience Mechanism:** We do not save an event immediately when a detection is missed (which could be due to camera vibration or temporary occlusion). We wait until `lost_frames` exceeds `PATIENCE_LIMIT` to confirm the pothole has truly left the scene.
- ◇ **Noise Filtering:** The `MIN_SAVE_AREA` check discards tiny, low-confidence detections that are often false positives.
- ◇ **Spatiotemporal Deduplication:** To prevent double-counting (e.g., if the vehicle stops and starts), we calculate the distance between the current event and `recent_saved` events. If it falls within `GPS_RADIUS`, it is marked as a duplicate and discarded.
- ◇ **Asynchronous I/O (Critical):** The actual disk write (image saving) and database insertion are heavy operations. To prevent FPS drops in the main inference loop, we simply push the data into `save_queue`. A separate background worker thread consumes this queue and handles the I/O.

A.4 Mobile Cloud Integration

A.4.1 Mobile UI Strategy

The Android application focuses on **Situational Awareness**:

- ◇ **Split-screen:** Simultaneously shows the road view (MJPEG) and spatial context (Google Maps).
- ◇ **Local Persistence:** Uses a local Room database to store history, ensuring functionality even in offline areas. Background workers handle synchronization when connectivity is restored.

A.4.2 Cloud Backend (Firebase)

A serverless architecture was chosen for rapid development and scalability.

- **Storage:** Hosts the evidence images.
- **Firestore:** specific collections (e.g., `potholes`) store metadata.
- **Security:** In production, API keys are restricted by SHA-1 fingerprint (Android) to prevent unauthorized usage.

A.5 Bill of Materials (BOM)

Table 6 outlines the estimated cost for reproducing the edge node.

Table 6: Indicative Hardware Cost Estimation.

Component	Functionality	Est. Cost (VND)
Raspberry Pi 4 Model B (4GB)	Main Compute Node	2.190.000
Pi Camera Module v3	Image Acquisition (CSI)	864.000
ATGM336H GNSS Module	Location Services (UART)	205.000
SD Card (64GB), Class 10	Storage	230.000
Total Estimated Cost		3.489.000